API documentation

for RoboCup SSL League Lab

October 20, 2026

1 Observations

The Strategy API represents the current state of the game using the `WorldState` data structure. It contains information about the ball, both teams of robots (Yellow and Blue), and the field geometry. This API will initially be used to develop and test strategies in **rSoccer**. In the future, some of your strategies will be adapted (through the API) to run in another simulator (**grSim**) allowing them to be tested in a different simulation environment.

1.1 `WorldState`

- **`ball`**: A `BallState` object describing the ball's current state. It is `None` if the ball is not detected.
- **`robots_blue`**: A mapping from robot IDs to `RobotState` objects for the blue team.
- **`robots_yellow`**: A mapping from robot IDs to `RobotState` objects for the yellow team.
- **`field`**: A `FieldGeometry` object containing the field dimensions.

1.2 `BallState`

The `BallState` data structure represents the ball's position and (if available) its planar velocity (along the x and y axis).

- **`x_m`**: Position of the ball along the x-axis, in meters.
- **`y_m`**: Position of the ball along the y-axis, in meters.
- **`vx_mps`** (optional): Velocity of the ball along the x-axis, in meters per second.
- **`vy_mps`** (optional): Velocity of the ball along the y-axis, in meters per second.

The velocity fields may be unavailable because they may not be provided by the simulator used (in such cases you could estimate the velocity of ball on your own). In rSoccer the velocities are generally provided.

1.3 `RobotState`

The `RobotState` data structure represents a robot's position, orientation, motion, and available sensor information.

- **`x_m`**: Position of the robot along the x-axis, in meters.
- **`y_m`**: Position of the robot along the y-axis, in meters.

- **heading_rad** (optional): Orientation of the robot, in radians.
- **vx_mps** (optional): Linear velocity along the x-axis, in meters per second.
- **vy_mps** (optional): Linear velocity along the y-axis, in meters per second.
- **angular_velocity_rad_s** (optional): Angular velocity of the robot, in radians per second.
- **velocity_source**: Indicates the source or availability of the robot's velocity. The value identifies whether velocity is measured (provided by the simulation environment),estimated (from the code by calculating its speed through frames), or unavailable.
- **infrared** (optional): Infrared sensor information, when provided by the simulator (it is provided by rSoccer). The infrared variable is boolean and indicates whether the ball is detected by the infrared sensor located near the robot's kicker, meaning the ball is close enough to be considered in the robot's possession. infrared = 1 means that the robot has possession of the ball.

All headings are expressed in radians. Velocities are expressed in meters per second, and angular velocity is expressed in radians per second.

1.4 FieldGeometry

The FieldGeometry data structure describes the dimensions of the playing field.

- **length_m**: Total length of the field along the x-axis, in meters.
- **width_m**: Total width of the field along the y-axis, in meters.
- **goal_width_m**: Width of the goal opening, in meters.

All three dimensions are required and are supplied by the simulator's field geometry.

2 TeamConfig

The TeamConfig data structure specifies which team the strategy controls, which robots it may command, and the direction of attack.

- **team**: Identifies the controlled team, either blue or yellow.
- **controlled_robot_ids**: A collection of robot IDs that the strategy is allowed to control.
- **attack_direction_x**: Specifies the team's attacking direction along the x-axis. A value of 1 means the team attacks toward positive x, while -1 means it attacks toward negative x.

The WorldState contains information about both teams. However, a strategy should return intents only for the robot IDs listed in controlled_robot_ids.

3 Commands

The strategy communicates its desired robot actions through the **RobotIntent** data structure.

A RobotIntent represents a requested action **for a single robot**. It specifies the desired linear and angular velocities, as well as optional ball-handling actions.

3.1 `RobotIntent`

- **`vx_mps`**: Requested linear velocity along the field's x-axis, in meters per second.
- **`vy_mps`**: Requested linear velocity along the field's y-axis, in meters per second.
- **`angular_velocity_rad_s`**: Requested angular velocity, in radians per second.
- **`kick_speed_mps`**: Requested kicking power in meters per second. It represents the kick speed and gets values from 0 to 5 m/s; 0 means no kick.
- **`dribbler`**: Boolean trigger indicating whether the robot's dribbler should be activated.

The linear velocities are expressed in the field coordinate frame, rather than as normalized policy values.

3.2 Example: Moving a Robot Toward a Target

The following example demonstrates how to generate an intent that moves robot 0 of the blue team toward the target position $(0, 1)$.

```
robot_id = 0
robot = state.robots_blue[robot_id]

target_x_m, target_y_m = 0.0, 1.0

dx = target_x_m - robot.x_m
dy = target_y_m - robot.y_m

distance = (dx * dx + dy * dy) ** 0.5

if distance <= 0.08:
    intent = RobotIntent()
else:
    speed_mps = min(1.0, distance)
    intent = RobotIntent(
    vx_mps=speed_mps * dx / distance,
    vy_mps=speed_mps * dy / distance
    )

return {robot_id: intent}
```

The target position is defined in meters. The robot's current position is subtracted from the target position to calculate the displacement along each axis.

The displacement is then normalized to obtain the movement direction. The requested speed is limited to 1.0 m/s and decreases as the robot approaches the target.

When the distance to the target is less than or equal to 0.08 meters, an empty `RobotIntent` is returned, indicating that no movement velocity is requested.

4 The Strategy Interface

The main task is to implement strategies through the `decide()` function of the Strategy class. Each strategy must implement the following method:

```
def decide(
self,
state: WorldState,
config: TeamConfig
```

```
) -> Mapping[int, RobotIntent]:
...
```

The function receives the current `WorldState`, which contains information about the ball, both teams of robots, and the field geometry. It also receives a `TeamConfig` that identifies the controlled team (Blue or Yellow) and the robot IDs belonging to that team (e.g. (0,1,2)).

Based on this information, the strategy must determine the appropriate action for each robot it controls and return a `RobotIntent` for each controlled robot, using the robot ID as the key in the returned mapping.

Each `RobotIntent` specifies the requested linear (ux, uy) and angular velocities, as well as whether the robot should activate its dribbler or kick the ball.

The strategy is responsible for deciding *what each robot should do*. The implementation of `decide()` should focus on decision-making and return the corresponding intents, without handling low-level communication or command transmission.

4.1 Motion Limits

The API uses physical units rather than normalized policy values.

The shared `MotionLimits` configuration defines the maximum permitted robot velocities.

- **Maximum planar velocity:** The default maximum is 2.0 m/s. The speed is limited by the magnitude of the velocity vector, $\sqrt{v_x^2 + v_y^2}$, rather than by limiting each component independently.
- **Maximum configurable angular velocity:** 3.0 rad/s.

The planar velocity is limited by its vector magnitude:

$$v = \sqrt{v_x^2 + v_y^2}$$

This means the combined linear velocity must respect the configured speed limit, rather than each velocity component being limited independently.

The angular speed cannot be configured above 3.0 rad/s.